Modelo de Datos

del banco de pruebas experimental

Modelado experimental de las fases prefill y decode de Qwen3-8B en NVIDIA A100 mediante vLLM

Código:	ITI-TFG-B26-0017
Autor:	José Andrés Ureta Granda
Tutor:	Andrès Xavier Rubio Proaño
Institución:	Universidad Tecnológica Indoamérica
	Facultad de Ingenierías — Ingeniería en TI
Documento:	Modelo de datos, versión 0.1 (borrador)
Fecha:	8 de octubre de 2026
Relacionado:	`SPEC.md` (secciones 8–12)



Índice

1. Propósito y alcance **2**

2. Convenciones **2**

3. Visión general **2**

4. Diagrama entidad–relación **4**

5. Diccionario de datos **5**

5.1. `entorno` . . . 5

5.2. `experimento` . . . 5

5.3. `config_servidor` . . . 6

5.4. `corrida` . . . 6

5.5. `peticion` . . . 7

5.6. `evento_token` . . . 7

5.7. `muestra_gpu` . . . 8

5.8. `muestra_vllm` . . . 8

5.9. `ajuste_modelo` . . . 9

5.10. `coeficiente` . . . 9

5.11. `ajuste_corrida` . . . 9

5.12. Vista `resumen_corrida` . . . 10

6. Métricas derivadas **10**

7. Reglas de integridad y calidad **11**

8. Almacenamiento físico **11**

9. Flujo de datos y linaje **12**

10. Ejemplo de registros **12**

11. Decisiones abiertas **13**

1. Propósito y alcance

Este documento define el modelo de datos que soporta la campaña experimental descrita en `SPEC.md`. Cubre la información que se captura, cómo se relaciona, con qué tipos y unidades se almacena y qué reglas de integridad deben cumplirse para que los modelos predictivos de prefill y decode sean trazables y reproducibles.

El modelo responde a cuatro necesidades:

1. **Contexto reproducible:** registrar el hardware, el software y la configuración de vLLM exactos de cada medición.
2. **Mediciones crudas:** tiempos por petición y por token, y telemetría de la GPU y del servidor.
3. **Métricas derivadas:** TTFT, TPOT/ITL, throughput, goodput, MFU y MBU por corrida.
4. **Modelos ajustados:** coeficientes, error de validación y qué corridas se usaron para entrenar y para probar.

Fuera de alcance: el texto de los prompts y de las respuestas generadas. Los prompts son sintéticos y se regeneran de forma determinista a partir de la semilla y de la longitud, y la calidad de las respuestas no se evalúa.

2. Convenciones

- **Nombres:** `snake_case`, en español y sin tildes. Las tablas van en singular.
- **Identificadores:** `UUID` (v4, texto) para las entidades que se generan en ejecución (`corrida`, `peticion`, `ajuste_modelo`). Para las entidades de contexto (`entorno`, `config_servidor`) el identificador es el `SHA-256` truncado a 16 caracteres de su contenido canónico en JSON: dos configuraciones idénticas producen el mismo identificador.
- **Tiempo:** las marcas de tiempo de alta resolución son `INT64` en nanosegundos de reloj monotónico (`time.perf_counter_ns`) y llevan el sufijo `_ns`. Las métricas derivadas van en milisegundos con el sufijo `_ms`. Las fechas de pared van en `TIMESTAMP` UTC.
- **Tokens:** siempre contados con el tokenizer oficial de Qwen3-8B.
- **Enumeraciones:** se almacenan como texto en minúsculas con valores cerrados (indicados entre llaves).
- **Tipos lógicos:** `TEXT`, `INT32`, `INT64`, `REAL` (float64), `BOOL`, `TIMESTAMP`, `JSON` (texto con JSON válido). Se mapean de forma directa a Parquet/Arrow y a DuckDB.
- **Leyenda:** **PK** = clave primaria; **FK** = clave foránea; los campos marcados con † son derivados (calculados en el análisis, no medidos).

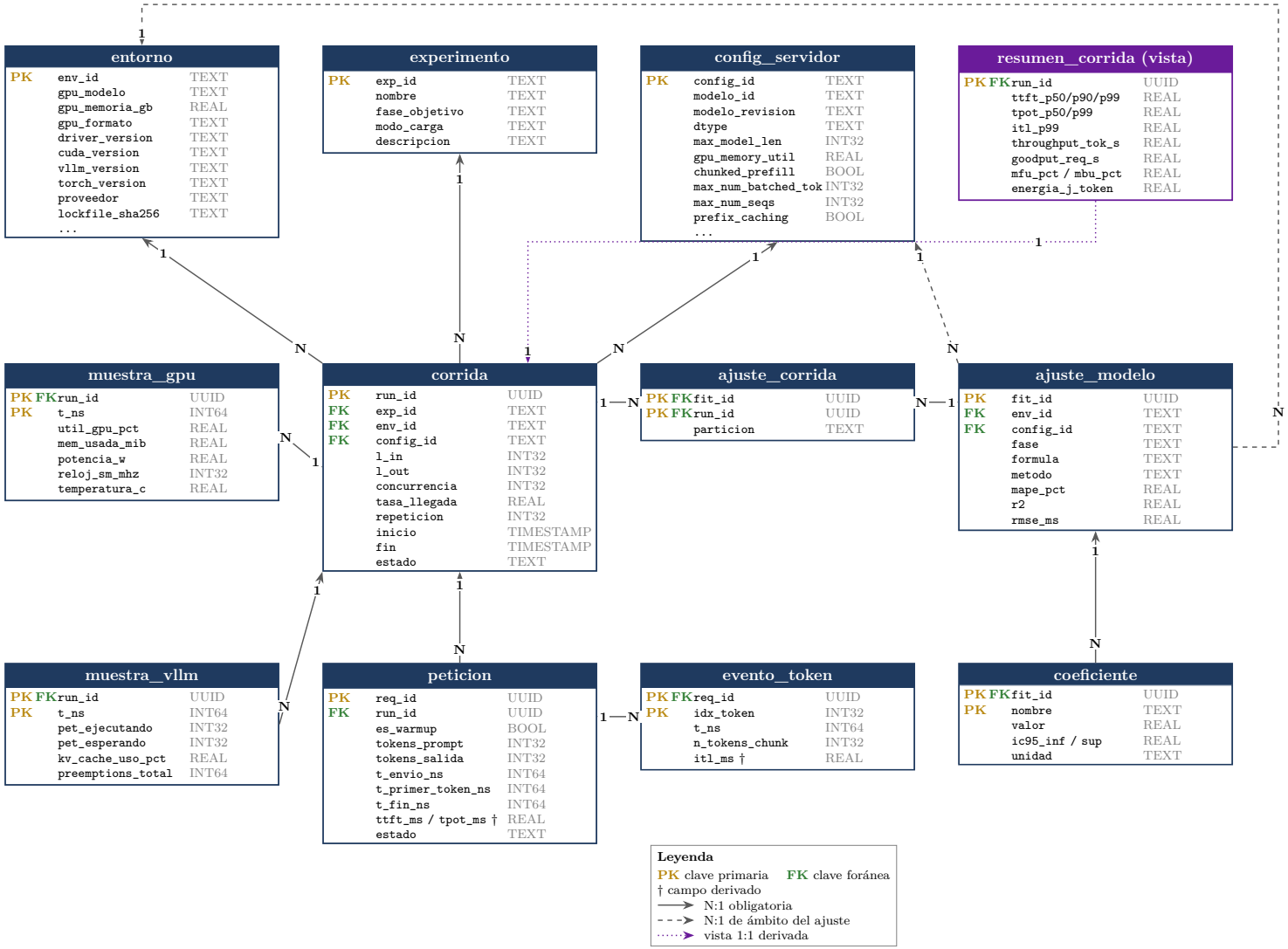
3. Visión general

El modelo se organiza en cuatro capas:

Capa	Entidades	Origen
Contexto	`entorno`, `config_servidor`, `experimento`	Registradas al inicio de la corrida
Ejecución	`corrida`, `peticion`, `evento_token`	Cliente de carga (`runner.py`)
Telemetría	`muestra_gpu`, `muestra_vllm`	`telemetry.py` (muestreo a 100 ms)
Modelado	`ajuste_modelo`, `coeficiente`, `ajuste_corrida`; vista `resumen_corrida`	`analysis/fit.py`, `load.py`

La entidad central es `corrida`: *un punto de la matriz experimental* (combinación de L_{in}, L_{out}, B, λ y configuración) *en una repetición concreta*. Todas las mediciones cuelgan de ella.

4. Diagrama entidad–relación



5. Diccionario de datos

5.1. `entorno`

Hardware y software exactos en los que se ejecutó una corrida. Se registra automáticamente desde `nvidia-smi`, `pip freeze` y el sistema operativo.

Campo	Tipo	Unidad	Nulo	Descripción
`env_id`	TEXT	—	no	**PK** Hash SHA-256 (16 car.) del JSON canónico de los demás campos.
`gpu_modelo`	TEXT	—	no	Nombre reportado, p. ej. `NVIDIA A100-SXM4-80GB`.
`gpu_uuid`	TEXT	—	no	UUID físico de la GPU; distingue tarjetas del mismo modelo.
`gpu_memoria_gb`	REAL	GB	no	Memoria total de la GPU.
`gpu_formato`	TEXT	—	no	{pcie, sxm4}.
`ancho_banda_pico_gbs`	REAL	GB/s	no	Ancho de banda HBM de la hoja de datos (para MBU).
`flops_pico_tflops`	REAL	TFLOPS	no	Pico BF16 Tensor Core denso (para MFU).
`driver_version`	TEXT	—	no	Versión del driver NVIDIA.
`cuda_version`	TEXT	—	no	Versión de CUDA.
`cpu_modelo`	TEXT	—	sí	Modelo de CPU del host.
`ram_gb`	REAL	GB	sí	RAM del host.
`so`	TEXT	—	no	Sistema operativo y kernel.
`python_version`	TEXT	—	no	Versión de Python.
`vllm_version`	TEXT	—	no	Versión exacta de vLLM.
`torch_version`	TEXT	—	no	Versión de PyTorch.
`imagen_contenedor`	TEXT	—	sí	Imagen Docker con digest, si se usa.
`lockfile_sha256`	TEXT	—	no	Hash del lockfile de dependencias.
`proveedor`	TEXT	—	no	Laboratorio o nube (p. ej. `gcp`, `runpod`, `lab-uti`).
`registrado_en`	TIMESTAMP	UTC	no	Primera vez que se observó este entorno.

5.2. `experimento`

Catálogo estático de los experimentos E1–E5 definidos en `SPEC.md` §8.2.

Campo	Tipo	Unidad	Nulo	Descripción
`exp_id`	TEXT	—	no	**PK** {e1, e2, e3, e4, e5}.
`nombre`	TEXT	—	no	Nombre corto (p. ej. “Prefill aislado”).
`fase_objetivo`	TEXT	—	no	{prefill, decode, e2e, interferencia, capacidad}.
`modo_carga`	TEXT	—	no	{offline, online}. *Offline*: lote fijo mediante la API `LLM`. *Online*: servidor compatible con OpenAI y llegadas de Poisson.
`descripcion`	TEXT	—	sí	Objetivo y diseño del experimento.

5.3. `config_servidor`

Parámetros con los que se lanzó vLLM. Cualquier cambio produce un `config_id` nuevo.

Campo	Tipo	Unidad	Nulo	Descripción
`config_id`	TEXT	—	no	**PK** Hash SHA-256 (16 car.) del JSON canónico de los parámetros.
`modelo_id`	TEXT	—	no	`Qwen/Qwen3-8B`.
`modelo_revision`	TEXT	—	no	Hash del commit de Hugging Face de los pesos.
`dtype`	TEXT	—	no	{bfloat16, float16}. Base: `bfloat16`.
`max_model_len`	INT32	tokens	no	Longitud máxima de contexto.
`gpu_memory_utilization`	REAL	fracción	no	Fracción de memoria reservada por vLLM (0–1).
`chunked_prefill`	BOOL	—	no	Factor experimental CP (on/off).
`max_num_batched_tokens`	INT32	tokens	sí	Tokens máximos por paso del planificador.
`max_num_seqs`	INT32	secuencias	sí	Secuencias concurrentes máximas.
`prefix_caching`	BOOL	—	no	Debe ser `false` en toda la campaña.
`enforce_eager`	BOOL	—	no	`true` desactiva los CUDA graphs.
`temperatura`	REAL	—	no	Muestreo; base 0 (greedy).
`ignore_eos`	BOOL	—	no	`true`, para forzar la longitud de salida exacta.
`semilla`	INT32	—	no	Semilla global.
`args_cli`	JSON	—	no	Línea de comandos completa, para auditoría.

5.4. `corrida`

Un punto de la matriz experimental en una repetición. Es la unidad mínima sobre la que se calculan los agregados.

Campo	Tipo	Unidad	Nulo	Descripción
`run_id`	UUID	—	no	**PK** Identificador de la corrida.
`exp_id`	TEXT	—	no	**FK** → `experimento`.
`env_id`	TEXT	—	no	**FK** → `entorno`.
`config_id`	TEXT	—	no	**FK** → `config_servidor`.
`l_in`	INT32	tokens	no	Longitud de entrada objetivo L_{in}.
`l_out`	INT32	tokens	no	Longitud de salida objetivo L_{out} (1 = prefill puro).
`concurrencia`	INT32	peticiones	no	Tamaño de lote / concurrencia B.
`tasa_llegada`	REAL	req/s	sí	λ de Poisson; nulo en modo offline.
`num_peticiones`	INT32	—	no	Peticiones medidas (sin contar el calentamiento).
`num_warmup`	INT32	—	no	Peticiones de calentamiento descartadas (≥ 5).
`repeticion`	INT32	—	no	Índice de repetición (1..R, $R \geq 5$).
`semilla_prompts`	INT32	—	no	Semilla del generador de prompts sintéticos.

Campo	Tipo	Unidad	Nulo	Descripción
kv_capacidad_tokens	INT64	tokens	sí	Capacidad de la caché KV reportada por vLLM al iniciar.
inicio	TIMESTAMP	UTC	no	Inicio de la corrida.
fin	TIMESTAMP	UTC	sí	Fin; nulo si la corrida se interrumpió.
estado	TEXT	—	no	{ok, oom, timeout, error, abortada}.
notas	TEXT	—	sí	Observaciones manuales.

Unicidad: (exp_id, env_id, config_id, l_in, l_out, concurrencia, tasa_llegada, repeticion).

5.5. peticion

Una solicitud de inferencia enviada durante una corrida. Las marcas de tiempo se toman en el cliente.

Campo	Tipo	Unidad	Nulo	Descripción
req_id	UUID	—	no	**PK**
run_id	UUID	—	no	**FK** → corrida.
indice	INT32	—	no	Orden de envío dentro de la corrida (0..n−1).
es_warmup	BOOL	—	no	true si es una petición de calentamiento (se excluye del análisis).
tokens_prompt	INT32	tokens	no	Tokens reales del prompt (reportados por vLLM en usage).
tokens_salida	INT32	tokens	sí	Tokens generados; nulo si la petición falló.
t_envio_ns	INT64	ns	no	Envío de la petición.
t_primer_token_ns	INT64	ns	sí	Recepción del primer token.
t_fin_ns	INT64	ns	sí	Recepción del último token.
ttft_ms †	REAL	ms	sí	$(t_{primer} - t_{envio})/10^6$.
tpot_ms †	REAL	ms	sí	$(t_{fin} - t_{primer})/\big(10^6 \cdot (\text{tokens_salida} - 1)\big)$; nulo si tokens_salida ≤ 1.
e2e_ms †	REAL	ms	sí	$(t_{fin} - t_{envio})/10^6$.
estado	TEXT	—	no	{ok, error, abortada, timeout}.
codigo_error	TEXT	—	sí	Mensaje o código HTTP del error.

5.6. evento_token

Cada fragmento (*chunk*) recibido por streaming. Permite calcular la distribución de ITL y detectar pausas causadas por el prefill de otras peticiones (E4).

Campo	Tipo	Unidad	Nulo	Descripción
req_id	UUID	—	no	**PK** **FK** → peticion.
idx_token	INT32	—	no	**PK** Índice acumulado del primer token del fragmento.

Campo	Tipo	Unidad	Nulo	Descripción
t_ns	INT64	ns	no	Instante de recepción en el cliente.
n_tokens_chunk	INT32	tokens	no	Tokens en el fragmento (puede ser > 1).
itl_ms †	REAL	ms	sí	$(t_{ns} - t_{ns}^{anterior})/(10^6 \cdot$ n_tokens_chunk); nulo para el primer fragmento.

5.7. muestra_gpu

Telemetría de la GPU muestreada cada 100 ms mientras dura la corrida (`nvidia-smi -query-gpu` o DCGM).

Campo	Tipo	Unidad	Nulo	Descripción
run_id	UUID	—	no	**PK FK** → corrida.
t_ns	INT64	ns	no	**PK** Instante de la muestra (mismo reloj que el cliente).
util_gpu_pct	REAL	%	no	Utilización de los SM.
util_mem_pct	REAL	%	no	Utilización del controlador de memoria.
mem_usada_mib	REAL	MiB	no	Memoria de GPU en uso.
potencia_w	REAL	W	no	Consumo instantáneo.
reloj_sm_mhz	INT32	MHz	no	Reloj de los SM (sirve para detectar *throttling*).
reloj_mem_mhz	INT32	MHz	no	Reloj de la memoria.
temperatura_c	REAL	°C	no	Temperatura de la GPU.
throttle_razones	TEXT	—	sí	Máscara o texto de las razones de *throttling*.

5.8. muestra_vllm

Estado interno del servidor, obtenido del endpoint Prometheus `/metrics` de vLLM (solo en modo online).

Campo	Tipo	Unidad	Nulo	Descripción
run_id	UUID	—	no	**PK FK** → corrida.
t_ns	INT64	ns	no	**PK** Instante del scrape.
pet_ejecutando	INT32	—	no	Peticiones en ejecución.
pet_esperando	INT32	—	no	Peticiones en cola.
kv_cache_uso_pct	REAL	%	no	Ocupación de la caché KV en la GPU.
preemptions_total	INT64	—	no	Contador acumulado de preemptions.
tokens_prompt_-total	INT64	tokens	no	Contador acumulado de tokens de prompt.
tokens_generados_-total	INT64	tokens	no	Contador acumulado de tokens generados.

5.9. `ajuste_modelo`

Un modelo predictivo ajustado para una fase, válido para un entorno y una configuración.

Campo	Tipo	Unidad	Nulo	Descripción
`fit_id`	UUID	—	no	**PK**
`fase`	TEXT	—	no	{prefill, decode, e2e, throughput}.
`env_id`	TEXT	—	no	**FK** → `entorno`. El modelo solo es válido en ese hardware.
`config_id`	TEXT	—	no	**FK** → `config_servidor`.
`formula`	TEXT	—	no	Forma funcional, p. ej. `a0 + a1*B*L + a2*B*L^2`.
`metodo`	TEXT	—	no	{ols, nnls, wls, por_tramos}.
`variable_respuesta`	TEXT	—	no	p. ej. `ttft_ms`, `t_paso_ms`.
`semilla_particion`	INT32	—	no	Semilla de la partición entrenamiento/prueba.
`frac_entrenamiento`	REAL	fracción	no	0,7–0,8.
`n_entrenamiento`	INT32	corridas	no	Corridas usadas para ajustar.
`n_prueba`	INT32	corridas	no	Corridas *hold-out*.
`mape_pct`	REAL	%	no	Error porcentual absoluto medio en prueba.
`r2`	REAL	—	no	Coeficiente de determinación en prueba.
`rmse_ms`	REAL	ms	no	Raíz del error cuadrático medio en prueba.
`rango_validez`	JSON	—	no	Rangos de L_{in}, B y C cubiertos por el entrenamiento.
`creado_en`	TIMESTAMP	UTC	no	Fecha del ajuste.

5.10. `coeficiente`

Campo	Tipo	Unidad	Nulo	Descripción
`fit_id`	UUID	—	no	**PK FK** → `ajuste_modelo`.
`nombre`	TEXT	—	no	**PK** p. ej. `a0`, `a1`, `b2`.
`valor`	REAL	var.	no	Valor estimado.
`error_estandar`	REAL	var.	sí	Error estándar de la estimación.
`ic95_inf`	REAL	var.	sí	Límite inferior del IC 95 %.
`ic95_sup`	REAL	var.	sí	Límite superior del IC 95 %.
`unidad`	TEXT	—	no	p. ej. `ms`, `ms/token`, `ms/token^2`.

5.11. `ajuste_corrida`

Tabla puente N:M que registra qué corridas se usaron en cada ajuste y en qué partición.

Campo	Tipo	Unidad	Nulo	Descripción
`fit_id`	UUID	—	no	**PK FK** → `ajuste_modelo`.
`run_id`	UUID	—	no	**PK FK** → `corrida`.
`particion`	TEXT	—	no	{entrenamiento, prueba}.

5.12. Vista `resumen_corrida`

Vista derivada (se materializa en Parquet y se recalcula con `analysis/load.py`). Contiene una fila por corrida con estado `ok`, calculada solo con peticiones `ok` y sin calentamiento.

Campo	Tipo	Unidad	Nulo	Descripción
`run_id`	UUID	—	no	**PK FK** → `corrida`.
`n_ok`	INT32	—	no	Peticiones válidas.
`ttft_media / p50 / p90 / p99`	REAL	ms	no	Estadísticos de `ttft_ms`.
`tpot_media / p50 / p99`	REAL	ms	sí	Estadísticos de `tpot_ms`.
`itl_p50 / p99`	REAL	ms	sí	Percentiles de `itl_ms` sobre todos los eventos.
`e2e_p50 / p99`	REAL	ms	no	Percentiles de `e2e_ms`.
`throughput_tok_s`	REAL	tok/s	no	Tokens de salida por segundo (ec. 1).
`throughput_req_s`	REAL	req/s	no	Peticiones completadas por segundo.
`goodput_req_s`	REAL	req/s	sí	Peticiones/s que cumplen el SLO (ec. 2).
`mfu_pct`	REAL	%	sí	Utilización de FLOPs; solo E1 (ec. 3).
`mbu_pct`	REAL	%	sí	Utilización de ancho de banda; solo E2 (ec. 4).
`util_gpu_media`	REAL	%	no	Media de `muestra_gpu.util_gpu_pct`.
`potencia_media_w`	REAL	W	no	Media de `muestra_gpu.potencia_w`.
`energia_j_token`	REAL	J/tok	no	Energía por token generado (ec. 5).
`preemptions`	INT64	—	sí	Diferencia del contador durante la corrida.

6. Métricas derivadas

Sea $\mathcal{P}$ el conjunto de peticiones válidas de una corrida (estado `ok` y sin calentamiento). Las constantes del modelo son: $N \approx 8{,}2 \times 10^9$ parámetros; $W \approx 16{,}4\,\text{GB}$ de pesos en BF16; $n_{capas} = 36$; $d = 4096$; caché KV de $k_{kv} = 2 \cdot 36 \cdot 8 \cdot 128 \cdot 2 = 147\,456\,\text{B}$ por token.

$$\text{throughput_tok_s} = \frac{\sum_{p \in \mathcal{P}} \text{tokens_salida}_p}{(\max_p t_{fin} - \min_p t_{envio})/10^9} \tag{1}$$

$$\text{goodput_req_s} = \frac{\left|\{p \in \mathcal{P} : \text{ttft}_p \le \text{SLO}_{ttft} \wedge \text{tpot}_p \le \text{SLO}_{tpot}\}\right|}{(\max_p t_{fin} - \min_p t_{envio})/10^9} \tag{2}$$

$$\text{MFU} = \frac{B\,(2NL_{in} + 2\,n_{capas} L_{in}^2\,d)}{T_{prefill} \cdot P_{pico}} \times 100 \tag{3}$$

$$\text{MBU} = \frac{W + B \cdot \bar{C} \cdot k_{kv}}{T_{paso} \cdot BW_{pico}} \times 100 \tag{4}$$

$$\text{energia_j_token} = \frac{\sum_i P_i\,\Delta t_i}{\sum_{p \in \mathcal{P}} \text{tokens_salida}_p} \tag{5}$$

En (3), $T_{prefill}$ se aproxima con la mediana de TTFT de E1, ya que $L_{out} = 1$, y el término de atención causal es una aproximación de primer orden. En (4), T_{paso} es la mediana de TPOT y $\bar{C}$ es

el contexto medio durante la generación ($L_{in} + L_{out}/2$). P_{pico} y BW_{pico} se toman de `entorno`. Los umbrales SLO se fijan en `SPEC.md` §16 (pendiente).

7. Reglas de integridad y calidad

R1. Integridad referencial: toda FK debe existir en la tabla padre. No se permiten huérfanos en `peticion`, `evento_token` ni en las muestras.

R2. Orden temporal: $t_{envio} \leq t_{primer\,token} \leq t_{fin}$, y los `t_ns` de `evento_token` son crecientes por petición.

R3. Longitudes exactas: en peticiones `ok`, tokens_prompt $= l_{in}$ y tokens_salida $= l_{out}$ (posible gracias a `ignore_eos`). Una diferencia invalida la corrida.

R4. Consistencia de streaming: $\sum$ n_tokens_chunk = tokens_salida por petición.

R5. Calentamiento: las peticiones con `es_warmup = true` nunca entran en agregados ni ajustes.

R6. Corridas válidas: solo las corridas con `estado = ok` y sin preemptions alimentan `resumen_corrida` y los ajustes (las que tienen preemptions se analizan aparte en E5).

R7. Aislamiento: si durante la corrida `muestra_gpu` registra memoria de procesos ajenos o *throttling* térmico, la corrida se marca en `notas` y se repite.

R8. Prefix caching: `config_servidor.prefix_caching = false` en toda la campaña. Los prompts de una misma corrida deben ser distintos entre sí (semilla + índice).

R9. Particiones disjuntas: dentro de un `fit_id`, ninguna combinación de factores aparece a la vez en entrenamiento y en prueba (todas sus repeticiones caen en la misma partición).

R10. Inmutabilidad: los datos crudos (`data/raw`) son de solo escritura una vez. Las correcciones se hacen repitiendo la corrida, nunca editando archivos.

8. Almacenamiento físico

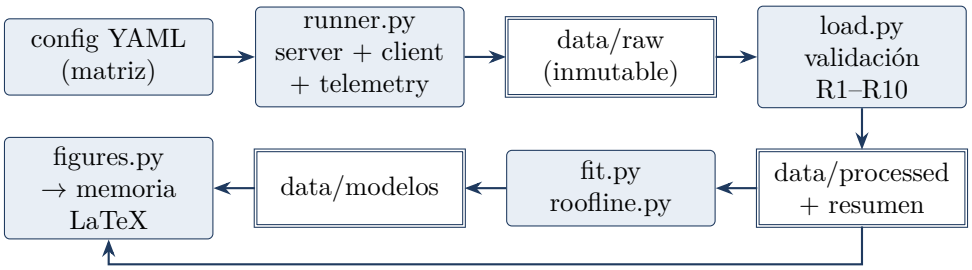
Los datos se persisten en **Apache Parquet** (columnar, con tipos y compresión `zstd`) y se consultan con pandas o DuckDB. No se requiere un servidor de base de datos.

```
data/
  raw/                                  # inmutable, una carpeta por corrida
    <exp_id>/<run_id>/
      manifest.json                     # corrida + config_servidor + entorno
      peticiones.parquet
      eventos_token.parquet
      muestras_gpu.parquet
      muestras_vllm.parquet             # solo modo online
      vllm_server.log                   # log completo del servidor
  processed/                            # regenerable con analysis/load.py
    entorno.parquet
    experimento.parquet
    config_servidor.parquet
    corrida.parquet
    peticion.parquet                    # particionado por exp_id
    evento_token.parquet                # particionado por exp_id
    muestra_gpu.parquet
    muestra_vllm.parquet
    resumen_corrida.parquet             # vista materializada
  modelos/                              # regenerable con analysis/fit.py
    ajuste_modelo.parquet
    coeficiente.parquet
    ajuste_corrida.parquet
```

Volumen estimado: la tabla más grande es `evento_token`. Con una matriz de unos 1 500–3 000 corridas, ∼100 peticiones por corrida y hasta 1 024 tokens por petición, se esperan del orden de 10^7–10^8 filas, unos pocos GB en Parquet comprimido. Esto es manejable en una sola máquina. Si el volumen fuera excesivo, en E1 y E5 se puede guardar solo el primer y el último evento.

Versionado: `data/raw` se respalda fuera del repositorio Git (es demasiado grande). En el repositorio se versionan los `manifest.json`, `processed/resumen_corrida.parquet` y `modelos/`, que son pequeños y suficientes para reproducir las figuras de la memoria.

9. Flujo de datos y linaje



Cada figura o tabla de la memoria incluye en su campo *Fuente* los `run_id` o el `fit_id` de origen (RNF4 del SPEC), con lo que se cierra el linaje desde el dato crudo hasta el documento.

10. Ejemplo de registros

Valores ilustrativos (no son mediciones reales).

corrida

run_id	exp_id	config_id	l_in	l_out	B	rep.	estado
`3f2a...`	e1	`9c01...`	2048	1	8	1	ok
`77b0...`	e2	`9c01...`	128	1024	64	3	ok

peticion

req_id	run_id	warmup	tok_prompt	tok_salida	ttft_ms	tpot_ms
a91e...	3f2a...	false	2048	1	412,7	—
c4d2...	77b0...	false	128	1024	61,3	14,9

ajuste_modelo y **coeficiente**

fit_id	fase	fórmula	método	MAPE (%)	R^2
e5f8...	prefill	a0 + a1·B·L + a2·B·L²	nnls	6,4	0,982

nombre	valor	unidad
a0	18,2	ms
a1	0,051	ms/token
a2	2,1e-6	ms/token²

11. Decisiones abiertas

- Umbrales SLO para el goodput (SPEC.md §16).
- Si en E1 y E5 se guardan todos los evento_token o solo el primero y el último (por volumen).
- Periodo de muestreo de la telemetría (100 ms propuesto; DCGM permite resoluciones más finas).
- Destino del respaldo de data/raw (disco institucional, Google Drive o almacenamiento en la nube).